

LABOUR-PARTY-2

Phase 0 — Native Android Architecture Blueprint

Kotlin · Compose · Material 3 · Clean Architecture · UDF · Hilt · Firebase

Commit audited: 2dd2fe4 · 2026-09-07

Architecture/specification only. **No application or Firebase resources implemented.**

Phase 0 Contents

[PHASE 0 - PRODUCT RECONCILIATION](#) (Part 1)

[PHASE 0 - ARCHITECTURE](#) (Part 2)

[PHASE 0 - RBAC & AUTH](#) (Part 3)

[PHASE 0 - FIRESTORE & RULES](#) (Part 4)

[PHASE 0 - SECURITY & TESTING](#) (Part 5)

[PHASE 0 - CONTRACTS & OFFLINE](#) (Part 6)

[PHASE 0 - PLATFORM & OPS](#) (Part 7)

ADR-001..015 (15 files)

PHASE 0 - PRODUCT RECONCILIATION

08-NATIVE-ANDROID/PRODUCT-RECONCILIATION.md

Product Reconciliation — Flutter reference → Native Android product

Status: Phase 0 reconciliation. Source: forensic audit (00-AUDIT-INDEX.md), code evidence, and the native product brief. Commit audited 2dd2fe4 . Classification: KEEP / IMPROVE / REPLACE / REMOVE / NEW / DEFER / UNVERIFIED. Open items are tagged **DECISION REQUIRED (D-xx)** per the Phase-0 rule (never silent ambiguity).

1. The three truths must stay separate

- **Flutter reference tells us:** what exists today — an offline, single-user, no-auth labour/trip recorder.
- **Product brief tells us:** what the native product should accomplish — a labour/workforce management app with Owner/Admin, Driver, Labourer roles and a secure backend.
- **Native architecture (this phase) defines:** how it should be built safely — reconciled below.

Do not mechanically translate Flutter code. Reconcile intent, then design.

2. Reconciled product definition (PROPOSED baseline)

Labour Party (native) = a labour/workforce & trip-management application for a **single-owner contracting business** (owner: Ramesh Sahu). The owner (and staff admins) records a day's **work sessions**, each containing **trips** (tractor + driver) and **labour attendance**; the system keeps authoritative, secure, cloud-backed records while remaining **usable offline in the field**, and (optionally) lets **drivers and labourers** log in to see their own assigned trips / attendance.

Scope boundary: labour/workforce operations only. No payroll/billing engine in v1 (payments DEFER). No unrelated features.

3. Reconciliation of core domain (existing → native)

Flutter entity/behaviour	Evidence	Native mapping	Class
Work (date + session + workType + place)	<code>work.dart</code>	<code>WorkSession</code> (daily unit the owner opens/closes)	IMPROVE
Trip (tractor, driver, tripNumber, notes, time)	<code>trip.dart</code>	<code>Trip</code> under a <code>WorkSession</code> ; tractor/driver become references (master) not free text	IMPROVE
Sequential trip numbering (morning/evening, per date)	<code>CalculateNextTripNumberUseCase</code>	Keep as server-enforced use case/rule	KEEP (move to backend authority)
Labour master list (name, optional phone)	<code>labour.dart</code>	<code>Labourer</code> catalogue (org-owned); optional link to a login account	IMPROVE
Driver as free-text name on trip	<code>trip.driverName</code>	<code>Driver</code> catalogue (org-owned) + optional user account link; trip references driver	REPLACE (free-text → master + picker)
Per-trip attendance present/absent	<code>TripLabour</code>	<code>Attendance</code> (server-authoritative, auditable, with history)	KEEP / IMPROVE
	<code>dashboard_screen.dart</code>	Owner dashboard	IMPROVE

Flutter entity/behaviour	Evidence	Native mapping	Class
Dashboard today (summary, quick-add next trip)			
Search today + filter (partial)	bloc events	Full search/filter UI + history-wide search	IMPROVE (finish what Flutter left partial)
History grouped date/session	history_screen.dart	History/archives screen	KEEP
Analytics KPIs + sortable table	analytics_screen.dart	Reports (owner)	IMPROVE
Local .labourbackup backup/restore	settings_screen.dart	Keep portable export (improved/encrypted) + cloud backup	IMPROVE
Draft autosave	draft_model.dart	Local drafts (Room/DataStore)	KEEP
Offline-first local (Hive)	Hive boxes	Hybrid: Room cache + Firestore offline persistence (cloud-authoritative)	REPLACE (see OFFLINE-SYNC)
Single local user, no auth	whole app	Authenticated roles	REPLACE
No remote backend	manifest (no INTERNET)	Firestore + Auth + Functions + FCM	NEW

4. Feature-class reconciliation

Feature	Flutter	Native class	Notes
Log daily work/trips	F-06..F-09	KEEP/IMPROVE	owner-authoritative; offline queue
Labour & attendance	F-10..F-12	KEEP/IMPROVE	auditable; server rule enforced
History/analytics	F-15/F-16	KEEP/IMPROVE	→ Reports
Backup/restore	F-17/F-18	IMPROVE	encrypted + cloud
Auth	none	NEW	required
Roles/RBAC	none	NEW	required (Driver/Labourer)
Owner account (Ramesh Sahu)	absent from code	NEW (provisioning, no hardcoded creds)	required
Driver self-service (see assignments)	none	NEW (scope dependent)	D-1
Labourer self-service attendance	none	NEW (scope dependent)	D-1
Job assignment & status workflow	none	NEW (owner→driver assignment is a scope decision)	D-1/D-6
Notifications	none	NEW	Phase later
Reports service / payments	none (light analytics)	DEFER	payments out of v1
Multi-organization/multi-owner	none	DEFER	single-org v1 (D-2)
Tractor/vehicle master	free text chips	IMPROVE	master list
Web/desktop companion	none	DEFER	native core only
Live map/GPS trip tracking	none	DEFER	not evidenced by product need

5. Screen reconciliation (existing → native)

Every Flutter screen becomes a native screen spec in `NATIVE-NAVIGATION.md`. Summary mapping:
 Splash→AuthGate/Splash; Dashboard→Owner Home; Trip Details→Trip detail (attendance editor); Add/Edit

Work→Work session / trip editor; Confirm Next Trip→copy-next-trip confirm; History→History; Analytics→Reports; Settings→Settings & Account; `/details` orphan→remove or become owner day-detail.

6. Explicit removals / don't-carries

- REMOVE: the orphan `/details` route + inert search/filter controls (BUG-02/03/04).
- REMOVE: free-text driver/tractor typos (replaced by catalogues + pickers).
- REMOVE: offline-first-as-only-mode architecture (becomes hybrid with cloud authority).
- REMOVE/DEFER: `glassmorphism` heavy decoration (native design system).
- DEPRECATED (flag): duplicate `LabourFormModel`, duplicate `@override`, dead switch branches — do not reproduce.

7. Evidence gaps / UNVERIFIED

- Whether real production data exists in any deployed Flutter install → affects MIGRATION-STRATEGY (D-3).
- Owner identity "Ramesh Sahu" vs package id `com.roshan.labourparty` → D-4.
- Session-boundary semantics (code 04:00 vs PRD text 00:00) → D-5.
- Whether drivers/labourers operate their own devices at all → D-1.

8. Decision register (Phase 0)

ID	Question	Recommended	Status
D-1	Do DRIVER and LABORER roles log in via their own app in v1, or are they owner-managed records only (with role model prepared)?	Owner runs core; DRIVER self-service optional; LABORER self-service likely DEFER. Role + data model prepared either way.	DECISION REQUIRED (business/ops)
D-2	Single organization vs multi-tenant?	Single-owner org in v1; multi-tenant deferred	DECISION REQUIRED
D-3	Migrate existing Hive/labourbackup production data?	Assume none unless told; provide export→import path if needed	DECISION REQUIRED
D-4	Final product/package identity & owner branding	Resolve before release	DECISION REQUIRED
D-5	Session boundary value	Follow product intent; fix inconsistency	DECISION REQUIRED
D-6	Job "assignment & status to driver" granularity	Owner-centric trips + attendance; driver assignment UI later	DECISION REQUIRED
D-7	Reports scope & KPIs v1	Owner reports (trips, attendance, driver/labourer summaries)	PROPOSED (confirm)
D-8	Retention/compliance	Not inventing legal; flag for confirmation	DECISION REQUIRED

9. Verification

Reconciliation grounded in code/audit (VERIFIED) for existing behaviour; native product targets are PROPOSED and tied to D-1..D-8. No implementation performed.

PHASE 0 - ARCHITECTURE

08-NATIVE-ANDROID/ANDROID-ARCHITECTURE.md

Native Android Architecture

Status: PROPOSED (Phase 0). Supersedes the earlier `ANDROID-ARCHITECTURE.md` outline with the reconciled spec. Dependency direction: **Presentation** → **Domain** ← **Data**. Domain must not depend on Firebase/Android.

1. Stack (baseline) — verify versions before implementation (see VERSION STRATEGY & DEPENDENCY-POLICY)

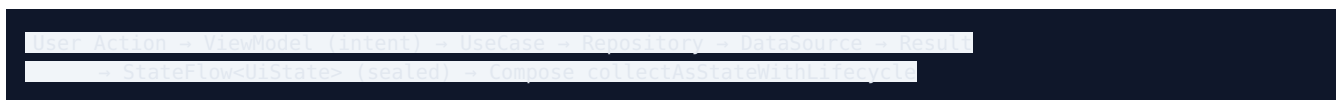
Kotlin + Jetpack Compose + Material 3 (stable, not alpha) + Navigation Compose + ViewModel/Lifecycle + StateFlow/Flow (UDF) + Clean Architecture + Hilt + Coroutines + Repository + Room (only where justified) + DataStore + Coil + WorkManager + Firebase (Auth, Firestore, Storage?, Cloud Functions, FCM, App Check, Crashlytics; optional Analytics/Remote Config).

2. Layer responsibilities



Rule: UI never calls Firebase/Room/business logic directly. UseCase orchestration in ViewModel. Repository is the seam for local/remote.

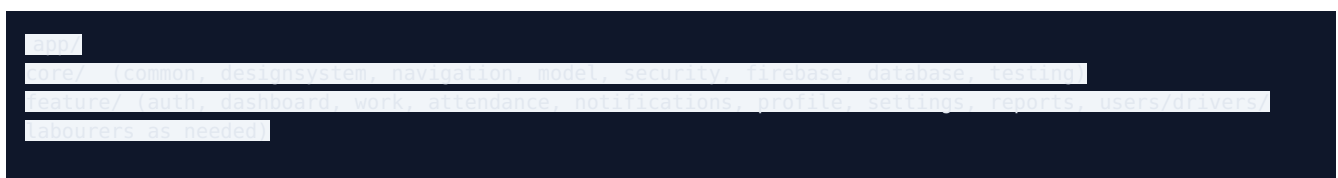
3. State management (UDF / MVI)



Explicit states per screen/model: Initial · Loading · Success · Empty · Error · Offline · Unauthorized · Forbidden · Refreshing · Submitting · SuccessWithStaleData . - No arbitrary global mutable state; scoped ViewModels (SavedStateHandle) survive config/process recreation. - One-way navigation handled explicitly (see ADR-013); avoid navigation-as-state where avoidable.

4. Module structure (avoid over-engineering)

Recommended single `:app` with clear package layering for an MVP **or** light modularisation:



```
domain/ (model, repository, usecase)
src/ (firebase, ui, ui-state, ui-view-models)
```

Decision: For a product of this size, **prefer a single app module with package boundaries**; split into modules only when compile/test/team size justify it. Do not force the example tree.

5. Concurrency & lifecycle

- Coroutines; `Dispatchers.IO` for DB/network; `Dispatchers.Default` for CPU; main-safe ViewModels; Flow collection in UI lifecycle-aware.
- WorkManager for reliable background (offline queue replay, backup, notification refresh).
- Repository operations suspend; datasources are the only place touching Firebase/Room.

6. Error & failure mapping

- Repository returns domain results mapping failures (network, auth, permission/Forbidden, offline, validation) — not raw Firebase exceptions leaking to UI.
- UI state carries typed error so the layer can offer retry/recovery.

7. Roles & authorization seam

- `AuthRepository` exposes `StateFlow<AuthUser?>` (uid, role). ViewModel uses role only to choose which **screens/actions** to show.
- Every sensitive op still goes to a server-enforced boundary (rules/CF). Never a client-only authorization decision.

8. Testing hooks

- Repository interfaces → fake/mock repositories in tests only.
- DI (Hilt) allows swapping Firebase datasources for emulator/local in tests.
- ViewModels unit-tested via use-case fakes; UiState pure.

9. Anti-goals

No business/backend logic in composables; no client-only authz; no global mutable state; no main-thread I/O; no network on main; no over-modularization.

08-NATIVE-ANDROID/DOMAIN-MODEL.md

Domain Model

Status: PROPOSED (Phase 0). Domain entities are **independent of Firebase/Room**. Model boundaries: Domain Model / Firebase Document Model / UI Model / DTO. Firestore documents must NOT become domain models directly.

1. Model boundary mapping

Model	Purpose	Direction	Notes
Domain Model	business invariants, role-agnostic	core	no Android/Firebase imports
DTO / DataTransfer	wire/DB payload (Firestore doc, Room entity)	data layer	maps to/from domain
Firebase Document Model	Firestore fields	data/remote	mappers
Room Entity	local cache	data/local	mappers

Model	Purpose	Direction	Notes
UI Model / UiState	Compose presentation	presentation	built in ViewModel from domain/state

Mapping is explicit via mappers in the data layer; **never pass Firestore DocumentSnapshot / Map into domain or UI.**

2. Domain entities

Personas/role metadata

- `UserAccount { uid, orgId, role(Role), status(AccountStatus), displayName, phone?, email?, photoUrl?, linkedDriverId?, linkedLabourerId?, timestamps }`
- `Role` enum: OWNER, ADMIN, DRIVER, LABORER.
- `AccountStatus`: INVITED, ACTIVE, SUSPENDED, DELETED(deactivated).
- `Driver { id, orgId, name, phone?, vehicle?, uid?, active }`
- `Labourer { id, orgId, name, phone?, uid?, active }`

Work/trip

- `WorkSession { id, orgId, date, session(Session), workType, place, status(OPEN/CLOSED), tripCount, timestamps }`
- `Session` enum: MORNING, EVENING (boundary value per D-5).
- `Trip { id, workSessionId, orgId, tripNumber, vehicleRef?, driverId?, driverNameSnapshot, status(TripStatus), notes?, place?, timestamps }`
- `TripStatus`: PLANNED, CONFIRMED, IN_PROGRESS, COMPLETED, CANCELLED (drive by D-6; keep minimal if no driver workflow).
- `AttendanceRecord { id, tripId, orgId, labourerId, status(PRESENT/ABSENT/CONFIRMED), recordedBy, timestamps }`

Notifications

- `NotificationItem { id, userId, type, title, body, deepLink?, read, createdAt }`
- `NotificationPreferences { userId, enabledChannels }`

Audit

- `AuditEvent { id, orgId, actorUid, action, targetType, targetId, previous?, next?, reason?, createdAt }`

Settings/org

- `Organization { id, name, ownerUid, settings }`
- `BusinessSettings` (session boundary, defaults, reports).

3. Domain invariants (business rules) — from audit (BR register)

- A `WorkSession` is uniquely `(org, date, session)`.
- Trip belongs to exactly one `WorkSession`; `tripNumber` sequential per session/date (server-authoritative).
- Attendance belongs to one Trip + one Labourer; status transitions restricted by role/state.
- Labourer names persist as typed; driver/labourer are master-referenced (improved).
- Destructive delete is soft + audited; a closed session is immutable-ish.
- Role/status changes only via server (CF). (Extend in `BACKEND-CONTRACT.md`.)

4. What is deliberately NOT a domain entity

- Cloud function payloads, FCM message envelopes, Firestore internal metadata, UI-only flags (e.g., `isLoading`) — stay in their layers.

5. Ownership (feeds rules)

Record → owner: WorkSession/Trip/Attendance/Drivers/Labourers → org; UserAccount/profile → user (+ org admin for role/status); Notification → recipient; AuditLog → system; Settings → org.

6. Verification

PROPOSED. Names preserved from audited Flutter domain where the concept carries (Work→WorkSession, Labour→Labourer, Trip→Trip, attendance), with REPLACE/IMPROVE noted in PRODUCT-RECONCILIATION.

08-NATIVE-ANDROID/NATIVE-NAVIGATION.md

Native Navigation

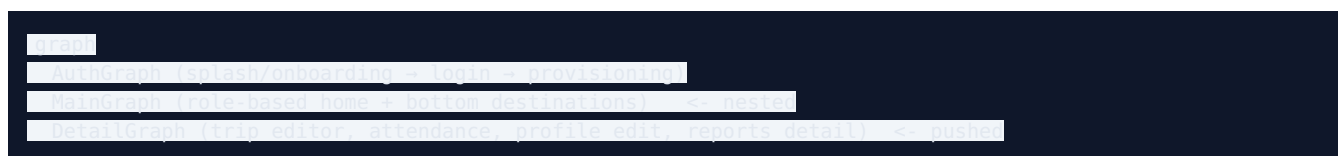
Status: PROPOSED (Phase 0). Determined from audited workflows, not forced names. Uses Navigation Compose; role-driven; adaptive on large screens.

1. Primary destinations

Current workflows (single-operator day entry + review + reports + settings). For the **OWNER/ADMIN** on a phone, recommended 4 primary destinations (≤ 5 guideline): 1. **Dashboard** — today's session(s), quick summary, quick-add. 2. **Work / Trips** — pick/date session, list trips, add/edit trip, attendance editor. 3. **Reports** — history/analytics KPIs, driver/labourer summaries. 4. **More / Menu** — crew (drivers/labourers), settings, account, backup/export, notifications badge. (Destinations are working names pending D-1/D-6; crew/reports placement may fold.)

DRIVER (if self-service, D-1): My Trips → My Attendance → Profile/Settings (small). **LABORER** (if self-service, D-1): My Attendance → Profile. FCM notification → surfaces a top-bar badge + notification centre (self) reachable from More.

2. Structure & Back behaviour



- **Back:** predictable; from detail returns to originating list; from a tab returns/updates; Back at root exits (or to overview per Android guidance). Predictive-back supported.
- **Logout:** sign out → AuthGraph; confirm; offline outbox preserved with notice (not silently discarded).
- **Session expiry / Forbidden:** intercept → re-auth screen with clear message; never drop data.
- **Deep links:** navigation handles typed deep links (job/trip/attendance) but each target re-validates access via rules; extra-param passing (Flutter weakness BUG-17) replaced by stable IDs loaded via repository, never whole-object extras.

3. Large screens (tablets/foldables)

- Use `MaterialWindowSizeClass` /adaptive: NavigationRail on medium+; two-pane master-detail for Work lists→editor and Reports.
- Do not force phone bottom-bar layout onto tablets/foldables.
- Compose Material3 adaptive only where stable.

4. Screen reconciliation output

Each Flutter screen → native spec (see PRODUCT-RECONCILIATION + screen-by-screen in this doc's companion [IMPLEMENTATION-ROADMAP.md](#) Phase 1..). Provide loading/empty/error/offline/success states per screen at implementation.

5. Open decisions

D-1 (role home sets), D-6 (driver workflow). Choose the most-restrictive safe nav until resolved.

6. Verification

PROPOSED. ADR-013.

08-NATIVE-ANDROID/ANDROID-DESIGN-SYSTEM.md

Native Android Design System (specification)

Status: PROPOSED design spec (Phase 0) — Jetpack Compose + Material 3 **stable (1.4.0)**; do not adopt 1.5.0-alpha. Visual target: premium, modern, polished, consumer-grade (no adult content; premium refers to hierarchy, media, cards, CTAs, motion, discovery). This is a custom product design system **on top of** Material 3 (extend/customize), not a replacement.

1. Brand & identity

- Brand: "Labour Party" (resolve identity/package with owner — OQ-2).
- Logo: current [app_icon_192.png](#) as source; create adaptive launcher icon + in-app mark.
- Iconography: Material Symbols/Outlined consistent set; brand-only accents.

2. Color system

- Build from brand seed (current primary [#246BFD](#) -like blue → cyan accent [#00D2FF](#)) via [dynamicLightColorScheme/dynamicDarkColorScheme](#) or fixed [MaterialTheme](#) .
- Semantic: success (present/attendance), error, warning, info mapped to M3 containers.
- Dark + light schemes; optional dynamic color (from wallpaper) behind a setting.
- Provide contrast-AA variants; never rely on alpha-38/54 for body text.

3. Typography

- Bundle fonts (Poppins for display/titles + Inter or similar for body) — no runtime fetch.
- M3 type scale (display/headline/title/body/label) mapped; line-height & tracking sane; supports large font scale.

4. Shape / spacing / elevation

- Shape scale: small 8 / medium 12 / large 16 / extra-large 24 (cards 16/24).
- Spacing: 4/8/12/16/24/32 system; page gutters 16.
- Elevation via M3 tonal + shadow tokens; elevation is semantic not decorative.

5. Core components

- Surfaces/cards (Card, tonal/elevated/outlined), Buttons (Filled/Outlined/Tonal/Text), TextFields (Outlined, filled), Dialogs & Sheets (bottom sheets for filters), TopBar, BottomBar + NavigationRail (tablet), Chips,

Badges, Lists (LazyColumn w/ keys), Loading (Linear/Circular + skeletons), Empty/Error states with retry, Snackbars, Swipe/Undo actions.

6. Motion & transitions

- Material motion tokens; shared-axis for screens; content fade/scale; keep subtle; honor `reduced motion`.

7. Responsive / large screen

- Phone canonical; width-based adaptive: use `BoxWithConstraints` / `MaterialWindowSizeClass`; NavigationRail + master/detail on tablets; multi-pane owner dashboard where valuable.

8. Accessibility

- Semantics everywhere, 48dp targets, AA contrast, large-font safe, reduced motion, TalkBack announcements (see ACCESSIBILITY.md).

9. Theming file layout (Compose)

`designsystem/theme/{Color,Type,Shape,Elevation}.kt`, `Theme.kt` (light/dark/dynamic), reusable component composables in `designsystem/components`.

10. Do-not

- No sexual/adult content; no imitation of such sites; keep a legitimate labour-management aesthetic: strong hierarchy, polished cards, clear CTAs, smooth transitions, good discovery, personalization within role.

11. Senior UX design review (Phase-0 §40)

Question	Answer/commitment
Is the primary action obvious?	Each screen has one primary CTA (e.g., dashboard "Record trip"); hierarchy reinforces it
Can users understand a screen within seconds?	Summary-first layouts; familiar M3 patterns
Is navigation predictable?	Role-driven bottom bar (≤ 5), consistent Back, master-detail on large screens
Is back behaviour consistent?	Yes — detail returns to origin; predictive-back; defined for logout/session-expiry
Are destructive actions protected?	Confirm + (owner-only) + audit + soft-delete + undo where reversible
Are loading states informative?	Skeleton/spinner + progress; offline indicator distinct
Are empty states useful?	Helpful empty states with a next-step CTA (fix Flutter blank-CTA bug)
Are errors recoverable?	Typed states with Retry (fix no-retry gap)
Is role-specific UI clear?	Role drives available actions; forbidden paths show clear Forbidden state, never fake success
Is information density appropriate?	Owner dashboards rich; labourer/driver minimal; adjustable reports
Does UI scale to larger devices?	Adaptive rails/two-pane; not phone layout stretched
Is accessibility preserved?	Semantics, 48dp targets, AA contrast, large-font, reduced-motion
Are animations purposeful?	Yes — only aid comprehension/feedback; honor reduced-motion

PHASE 0 - RBAC & AUTH

08-NATIVE-ANDROID/RBAC-DESIGN.md

RBAC Design

Status: PROPOSED (Phase 0). Enforcement model: client renders by role; **authorization is enforced in Firestore Security Rules + Cloud Functions** (FIREBASE-SECURITY-RULES-DESIGN.md). Cell values: ALLOW / DENY / CONDITIONAL / NOT APPLICABLE. D-1 (labourer login scope) pending — matrix assumes all roles exist.

1. Roles

Role	Meaning
OWNER	Business owner (Ramesh Sahu) — highest privilege, business identity, provisioned server-side, never hardcoded
ADMIN	Delegated operator within the owner's org (manages day ops)
DRIVER	Tractor/driver operator (performs trips)
LABORER	Field worker (attendance subject)

ADMIN optional in v1 (D-2 scope); OWNER and ADMIN share an admin capability set where ADMIN is a scoped delegate.

2. Capability matrix

Legend: ☐ ALLOW · ☐ DENY · ☒ CONDITIONAL (condition noted) · ☐ NOT APPLICABLE

Capability	OWNER/ADMIN	DRIVER	LABORER
Login	☐	☐	☐ (requires account; D-1)
View own profile	☐	☐	☐
Edit own profile (own name/phone/photo)	☐ (not role/email)	☐ (not role/email)	☐ (not role/email)
View dashboard	☐	☒ driver dashboard (own assigned trips)	☒ labourer summary (own attendance) if self-service
Manage users/accounts	☐ (OWNER) ; ADMIN ☒ if delegated	☐	☐
Manage drivers (catalogue)	☐ (OWNER); ADMIN ☒	☐	☐
Manage labourers (catalogue)	☐ (OWNER); ADMIN ☒	☐	☐
Create work session	☐	☐	☐
Assign work/labour/driver to trips	☐	☐	☐
View assigned work	☐ all	☐ own (CONDITIONAL by driverId)	☐ own attendance (if self-service)
Update work/trip status	☐	☒ confirm/completion of own assigned trip only (if driver workflow)	☐
Record/confirm attendance	☐ (record)	☒ confirm own trip attendance? (recommend: DENY direct; owner/CF records)	☐ view own; ☐ modify others

Capability	OWNER/ADMIN	DRIVER	LABORER
Edit completed work	👤 (owner only, audited, restricted window)	👤	👤
Delete records	👤 (owner; audited; soft-delete)	👤	👤
View reports	👤	👤 own-trip summary only	👤 (own attendance only)
Manage notifications	👤 (owner sends)	receive own	receive own
Notification preferences	👤	👤	👤 (if account)
System/business settings	👤 (OWNER)	👤	👤
Audit logs	👤 (OWNER/ADMIN read)	👤	👤
Role changes / suspend users	👤 (OWNER; CF)	👤	👤
Account deletion	👤 own account self-delete (with safeguards); owner deletion protected	👤 own	👤 own
Data export	👤 (owner)	👤	👤 (own-data request)

3. Rule of three for enforcement

1. UI shows actions the role may take (routing/visibility).
2. Firestore/Storage **security rules** deny/allow every read/write by `request.auth.uid`, role, and ownership/org.
3. **Cloud Functions** perform operations the client must not do directly (role assignment, audit writes, notifications, authoritative numbering, destructive deletes).

4. Ownership primitives used in rules

- `orgId` on every business record.
- `assignedDriverUid` (or driver link) to scope driver reads/writes.
- `labourerUid` to scope labourer reads.
- `createdBy / changedBy = request.auth.uid`.
- Role stored in `users.role` (authoritative; rule checks `request.auth.uid → get users doc` or custom claim) — see ADR-010.

5. Anti-patterns (explicit)

- No `allow read, write: if request.auth != null;` as a whole-collection strategy.
- No trusting client `isAdmin`, `role`, `permissions`, `owner` fields as authorization.
- No unconditional cross-user reads/writes.

6. Open decisions

D-1 (driver/labourer self-login), D-2 (ADMIN delegation), D-6 (driver job-status workflow). Until resolved, security rules grant the most restrictive safe set.

08-NATIVE-ANDROID/FIREBASE-ARCHITECTURE.md

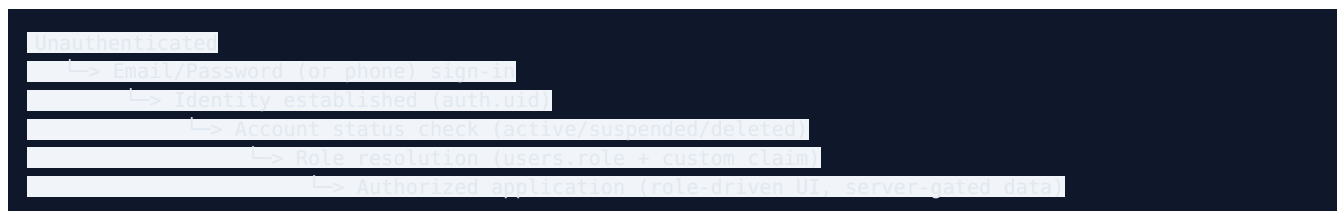
Firestore Architecture

Status: PROPOSED (Phase 0). This is the conceptual architecture; **no Firebase project/config is created** (D-4). Each service below states its responsibility and why (or why not) it is used.

1. Service responsibilities

Service	Responsibility	Decision	Why
Firebase Authentication	Identity + session lifecycle	☐ Use	email/password (owner/admin + any staff); phone evaluated for driver/labourer field users if self-service (D-1). See Authentication section.
Cloud Firestore	Primary cloud application data (authoritative)	☐ Use	org-scoped domain data, offline persistence, security rules
Cloud Storage	Only binary/file data needing cloud	⚖ Conditional	profile images, media, encrypted backup exports. Not needed if we defer photos
Cloud Functions	Trusted server-side ops only	☐ Use (selectively)	role assignment/transfer, audit writes, authoritative trip numbering, notifications, destructive ops, backup orchestration. Not a wrapper around every Firestore call — direct client+Rules is preferred
FCM	Push notifications	☐ Use (Phase later; enable in v1.x)	role-scoped assignment/attendance/admin notices
App Check	App authenticity / abuse protection	☐ Use	protect Firestore/Storage/Functions from non-app clients
Crashlytics	Crash monitoring	☐ Use	release quality
Analytics	Product analytics where justified	⚖ Optional/opt-in	only if product value confirmed; privacy-reviewed, non-PII
Remote Config	Legit remote config only	⚖ Optional	e.g., feature flags/thresholds if operationally valuable; do not add for show

2. Authentication architecture (identity → session → role)



Behaviour matrix: | Condition | Behaviour | |---|---| | Invalid credentials | Auth error → clear message; no account enumeration | | Disabled account | Status `suspended / disabled` → reject sign-in or restrict; message; support path | | Deleted account | Auth error "no account"; offer create/recover | | Network failure / Firebase outage | If offline & authenticated session valid → continue on local cache; if sign-in required & offline → message + retry (offline sign-in only if we adopt Firebase token persistence + cached rules decision; see OFFLINE-SYNC) | | Expired/revoked session | Force re-auth → sign-in; preserve offline queue; no data loss | | Unauthorized role | After auth, rules deny → Forbidden state; route to minimal profile/home; never silently grant | | Incomplete profile | On first login, if profile missing (orgId/role not yet linked) → provisioning/onboarding screen |

Decisions: - Enable **email/password** only for v1 unless D-1 adds phone. Do NOT auto-enable every provider (Google/Apple/Facebook not required). - **Password reset**: Firebase reset email — owner/admin + staff accounts. - **Email verification**: recommended for admin/owner identity. - **Account activation**: provisioning link/CF creates account with temp setup; **no hardcoded owner credentials**. - **Session**: Firebase token persistence; App Check attached; token refresh handled by SDK. - See ADR-009, `ANDROID-SECURITY-ARCHITECTURE.md`, `RBAC-DESIGN.md`.

3. Service boundary diagram



```
Auth --> FS[Firestore rules]
Auth --> GS[Storage rules]
CF[Cloud Functions]
CF --> Push[FCM]
CF --> Audit[(audit_logs)]
FS --> Crash[Crashlytics]
```

4. Explicit anti-patterns

- Cloud Functions are NOT a generic proxy for Firestore. Direct client reads/writes under well-tested Security Rules are the default; Functions only where trusted logic/rules can't express it.
- No Firebase service-account/admin credentials ever placed in the Android app.

5. Ownership/environment

No actual project IDs invented (D-4). Environment strategy in `CI-CD-ARCHITECTURE.md`.

PHASE 0 - FIRESTORE & RULES

08-NATIVE-ANDROID/FIREBASE-DATABASE-DESIGN.md

Firestore Database Design

Status: PROPOSED conceptual schema (Phase 0). No collections are created. Every proposed collection specifies owner, IDs, fields, indexes, lifecycle, PII, permissions, deletion policy, audit. Overlap with prior `FIREBASE-DATABASE.md` supersedes it.

Conventions

- Every business record: `orgId` (owner org), `createdAt` / `updatedAt` (server timestamps where possible), `createdBy` / `changedBy` = `request.auth.uid`, optional soft-delete `deletedAt` / `deletedBy`.
- PII minimised; only fields actually required.
- Single-org in v1 (D-2), schema keeps `orgId` so multi-org is a future-safe index key without over-building.

Collections

`organizations` (1 per owner business) — **DECISION: keep minimal for v1**

- Purpose: org root/tenant + business settings (offline-first scope). Owner: OWNER.
- Doc ID: fixed single doc (e.g., `orgs/single` in v1) or autoid.
- Fields: `name` (str req), `ownerUid` (req), `displaySettings{...}`, `active` (bool), timestamps.
- Indexes: none for v1 single-org; by `ownerUid` if needed.
- Permissions: read OWNER/ADMIN/(members scoped); write OWNER only.
- Deletion: never hard-delete org while records exist (soft-deactivate). Audit: settings changes.

`users` / `profiles`

- Purpose: identity + role + status + profile. Owner: the user (self) + org admin (role/status).
- Doc ID: `users/{authUid}` (1:1 with auth).
- Fields: `uid` (req), `orgId` (req), `role` (enum owner/admin/driver/labourer; req), `status` (active/suspended/invited; req), `displayName` (req), `phone` ?(opt), `email` ?(opt, masked in UI), `photoUrl` ?(opt), `driverId` / `labourerId` ?(opt link to catalogue), `createdAt`, `updatedAt`, `createdBy`.
- Indexes: `orgId+role`, `orgId+status`.
- PII: YES (name, phone, email). Minimise; phone only if needed for contact/login.
- Permissions: read self + OWNER/ADMIN (org); write self only limited profile fields (name/phone/photo) — role/status ONLY via CF.
- Deletion policy: deactivate (never hard-delete org-linked), account deletion via CF w/ audit.
- Audit: role/status/email changes.

`drivers` (catalogue)

- Purpose: master list of drivers (org-owned), referenced by trips. Owner: org/OWNER.
- Doc ID: autoid.
- Fields: `orgId`, `name` (req), `phone` ?(opt), `vehicle` ?(opt ref or text), `uid` ?(opt link to user account, if D-1), `active`, timestamps, `createdBy`.
- Permissions: read OWNER/ADMIN (drivers? maybe all if driver sees roster—DECISION), write OWNER/ADMIN.
- Deletion: soft (keep historical trip refs). Audit on edits.
- PII: name/phone.

labourers (catalogue)

- Purpose: master labour force (org-owned). Owner: org.
- Fields: `orgId` , `name` (req), `phone` ?(opt), `uid` ?(opt link if D-1), `active` , `dailyRate` ?(opt, owner-only) — DECISION whether wage stored; timestamps.
- Permissions: read OWNER/ADMIN; write OWNER/ADMIN.
- Deletion: soft. PII: name/phone. Audit: edits.

workSessions (mirrors Flutter Work)

- Purpose: a daily session unit (date + session morning/evening) an owner opens. Owner: org.
- Fields: `orgId` , `date` (req, as YYYY-MM-DD), `session` (enum morning/evening), `workType` , `place` , `status` (open/closed), `tripCount` , timestamps, `createdBy` .
- Indexes: `orgId+date` , `orgId+date+session` .
- Subcollection: `workSessions/{id}/trips` .
- Permissions: read OWNER/ADMIN (+ assigned drivers scope); write OWNER/ADMIN; close only by OWNER (or ADMIN if delegated).
- Deletion: soft only; a closed session is immutable-ish (audited).

trips (subcollection workSessions/{id}/trips)

- Purpose: one trip in a session. Owner: org (creator records).
- Fields: `orgId` (inherited), `tripNumber` (req, server/CF-assigned), `tractorRef` / `vehicle` ?(ref), `driverRef` (ref) OR `driverName` , `labourIds` ?(denormalised), `status` (enum draft/planned/confirmed/completed/cancelled per D-6), `notes` , `place` , timestamps, `createdBy` .
- Indexes: by `sessionId+tripNumber` .
- Permissions: read OWNER/ADMIN + (assigned driver if self-service); write OWNER/ADMIN; driver status update via CF only.
- Deletion: soft; preserve numbering invariants.
- Audit: creation/status changes.

attendance (or subcollection per trip)

- Purpose: per-worker per-trip participation/status. Owner: org + worker (subject).
- Recommend structure: `trips/{tripId}/attendance/{labourerId}` for easy per-trip query, plus roll-up counters on trip.
- Fields: `labourerId` , `status` (present/absent/confirmed), `isPresent` (bool derived), `recordedBy` , `confirmedByDriver` ?(bool, if D-1), timestamps.
- Indexes: `labourerId+date` for a worker's history (requires labourer→date index; use a separate `labourerAttendance` view or query path).
- PII: links to labourer.
- Permissions: write OWNER/ADMIN (record); read worker own (if self-service); labourer cannot modify another's.
- Audit: changes/corrections by OWNER only.

notifications (+ notificationPreferences)

- Purpose: user notification history + per-user preferences. Owner: recipient (history/prefs).
- Fields (notifications): `userId` , `type` , `title` , `body` , `data{deepLink}` , `read` (bool), `createdAt` .
- Fields (prefs): `userId` , `enabled channels`, quiet hours (optional).
- Indexes: `userId+read+createdAt` .
- Permissions: read/write self only; creation via CF/server only (never direct client broadcast).
- Deletion: retention window (D-8); read/unread update by owner.
- Audit: sends (server).

auditLogs

- Purpose: admin/privileged action log. Owner: system.
- Fields: `orgId` , `actorUid` , `action` , `targetType` , `targetId` , `previousState` (opt map), `newState` (opt map), `reason` ?(opt), `createdAt` (server).

- Permissions: write CF/admin only (rules deny direct client write); read OWNER/ADMIN.
- Retention: policy (D-8).
- Never log secrets.

`settings` / **business config**

- Purpose: org-level business settings. Owner: org/OWNER.
- Fields: session boundary, work types, default tractor/driver hints, reporting flags (only confirmed).
- Permissions: read members (scoped), write OWNER.

`notifications/readUnread` , `vehicles` **optional**

- Vehicle master optional (KEEP from reference: tractor picker).

Relationships & indexes summary

org → users; org → drivers/labourers; org → workSessions → trips → attendance; users ↔ driver/labourer (optional link). Composite indexes per query patterns listed. Consistency via batched writes/transactions for numbering & counters. Denormalise only with documented propagation.

Ownership feeding rules

`orgId` + `role` + `creator/assignee/labourer` fields drive every security rule (see `FIREBASE-SECURITY-RULES-DESIGN.md`).

Retention / deletion policy (D-8; confirm, don't invent)

Active → archive (closed sessions) → soft-delete with audit → scheduled purge per policy. Notifications retention window. Uploaded files lifecycle. Legal/regulatory flags to business.

Verification

PROPOSED only; no current Firestore schema exists (VERIFIED absence). D-1..D-8 control optional collections/fields.

08-NATIVE-ANDROID/FIREBASE-SECURITY-RULES-DESIGN.md

Firestore & Storage Security Rules — Design

Status: PROPOSED conceptual rules (Phase 0). No rules are deployed. These define WHO can do WHAT on WHICH doc under WHAT conditions. Production rules must be written and tested against the **Firestore Emulator** before deployment (Firebase recommendation).

1. Principles

- Default deny.
- Every sensitive op: (actor, verb, resource, condition).
- Authorization checks `request.auth.uid` , role (custom claim or `users` doc), and ownership (`orgId` , `creator` , `assignee` , `labourerId`). Role resolved server-side; never trust client fields.
- Never: `allow read, write: if request.auth != null;` as strategy.
- Cloud Functions for ops rules can't express (role change, audit write, destructive delete, notifications, authoritative numbering).

2. Helpers (conceptual)

```
function signIn(){return request.auth!=null;}
function isAdmin(orgId){
  return signIn() &&
  request.auth.token_org == orgId;
}
function isMember(orgId){ return isAdmin() && request.auth.token_org == orgId;
}
function isOwner(orgId){ return isMember() && request.auth.token_org == orgId;
}
```

3. Firestore rules (sketch — definitive version lives with implementation + emulator tests)

Resource	read	write	Condition notes
users/{uid}	self, or OWNER/ADMIN of org	create self; update self limited profile; role/status via CF	no client role change
drivers/labourers	OWNER/ADMIN (org)	OWNER/ADMIN	soft-delete only
workSessions	OWNER/ADMIN; drivers see own assigned	create OWNER/ADMIN; close OWNER/ADMIN(admin delegated)	
trips	OWNER/ADMIN; assigned driver (self-service)	create/update OWNER/ADMIN; driver status via CF	
attendance	OWNER/ADMIN; worker self	record OWNER/ADMIN; worker read-only	worker cannot modify others
notifications/{userId}	self only	self toggles read; server creates	
auditLogs	OWNER/ADMIN	CF/admin only (deny client)	
settings	members scoped	OWNER	

Example gating an owner admin user-management is done in a CF (not direct Firestore), because role/status changes + audit must be atomic and privileged.

4. Storage rules (sketch)

- Profile photos: `orgs/{orgId}/users/{uid}/...` — read members (scoped), write self (owner of that uid path) or OWNER/ADMIN. Allowlist content types/size.
- Encrypted backups/exports: `orgs/{orgId}/backups/...` — read/write OWNER/ADMIN only; App Check.
- Never a public bucket.

5. Emulator test matrix (must cover)

Owner CRUD; admin delegation; driver read-own-only; labourer read-own-only; labourer attempting to modify another's attendance = DENY; user attempting role escalation = DENY; cross-org access = DENY; unauthenticated = DENY; Storage unauthenticated/abuse = DENY.

6. App Check

Enforce App Check on Firestore/Storage/Functions. Handle attestation failures gracefully (deny, surface app-update message) without DoS of legitimate users.

7. Replay / abuse / rate limiting

Functions enforce quotas & idempotency; alerting on anomalies; no silent overwrite of user data.

8. Anti-patterns (explicit)

Never trust `isAdmin`, `role`, `permissions`, `owner` from client. No broad allow. No unbounded nested reads via rules without size guards.

Verification

Conceptual only; actual rules authored & tested in emulator during implementation. Security review (Phase-0 §46) enumerated in `ANDROID-SECURITY-ARCHITECTURE.md`.

PHASE 0 - SECURITY & TESTING

08-NATIVE-ANDROID/ANDROID-SECURITY-ARCHITECTURE.md

Android Security Architecture

Status: PROPOSED (Phase 0). Covers auth, authorization, RBAC, rules, App Check, secure storage, Keystore, network, logging, secrets, components, deep links, file access, permissions, screenshots, backups, deletion, privacy. Derived from the audit's security findings (SEC-1., SA-1, ST-*).

1. Foundational requirement (from audit)

Before any native release, remediate the **CRITICAL audit finding**: the committed `android/app/keystore.jks.bak` must be removed from history (branches + tags) and the signing key rotated. Do not carry this defect forward.

2. Layers

Layer	Mechanism
Authentication	Firebase Auth (email/password v1; phone if D-1)
Authorization	Firestore/Storage rules + CF; role server-authoritative (custom claim + <code>users.role</code>)
App authenticity	Firebase App Check (Play Integrity) on Firestore/Storage/Functions
Secure local storage	Android Keystore for app-managed secrets/tokens; encrypted storage for sensitive local data; plain DataStore only for non-sensitive prefs
Network	TLS 1.2+ via Firebase/default; network security config; no cleartext
Logging	structured, redact secrets/PII
App surface	minimal exported components; deep-link scheme allowlisted; safe intent handling

3. Secure storage / local data policy

- What may be in **DataStore**: non-sensitive preferences, last-known UI prefs, feature flags (from Remote Config).
- What needs **encrypted storage**: cached auth-adjacent tokens if persisted beyond SDK, encrypted offline backup key material, any sensitive cached records — evaluate EncryptedSharedPreferences/SQLCipher for cached PII (labourer names/phones) at rest.
- What must **never** be stored locally: Firebase service-account/admin credentials, other users' role-escalation data, raw passwords, bulk PII not required for function.
- Room stores the offline outbox/cache; encrypt sensitive columns/tables or the DB where justified (D-8 privacy).

4. Logout & account-switch cleanup

Sign-out clears: session token, outbox **with user consent** (or preserves per-account queues), cached other-user data, notification cache. Account switch isolates per-account data; never leak prior user's cache to next user. Cache invalidation on role/org change.

5. Keystore & crypto

- Android Keystore (non-exportable) for any signing/encryption keys; no keys in source or DataStore.
- No hardcoded credentials/owner account (Ramesh Sahu is a business identity, provisioned server-side).

6. Deep links / intents

- Deep links handled via Navigation; each deep link target re-validates the resource is visible to the current user (rules) before showing; never reveal data via link payload.
- No exported components beyond the launcher/main activity + deep-link handlers with allowlisted hosts.

7. Screenshots & privacy

- FLAG_SECURE only where genuinely sensitive screens require it (e.g., if showing other users' contact lists); not blanket (hurts UX). Decision D-8.

8. Backups

- Cloud (auto) backup: exclude sensitive local caches by default; evaluate per D-8. Offline `.labourbackup` -style export should be encrypted + authenticated.

9. Deletion & data privacy

- Account deletion via CF with audit + dependent-record handling; user data export request path; org data deletion policy (D-8). No silent purge.

10. Security review checklist (Phase-0 §46 — attack scenarios)

Must be tested in emulator/dev (never production): privilege escalation, IDOR (direct doc access by another uid), client-controlled role escalation, unauthorized reads/writes, cross-user data access, insecure Storage access, token misuse, account-takeover paths (password reset, replay), notification abuse, excessive permissions, sensitive local storage, debug leakage. Documented as test cases in `ANDROID-TESTING-ARCHITECTURE.md`.

11. Verification

Conceptual; actual config at implementation. Tie to ADR-007..010, `FIREBASE-SECURITY-RULES-DESIGN.md`.

08-NATIVE-ANDROID/ANDROID-TESTING-ARCHITECTURE.md

Android Testing Architecture

Status: PROPOSED (Phase 0). Defines a layered test strategy and mandatory Firebase-rules automated tests. Security-critical rules MUST be automated against the Emulator.

1. Test pyramid



Layer	Scope	Tech
Unit	pure domain, usecases, mappers	JUnit
Domain/Repository	business rules w/ fake repos	JUnit + fake/mock (test-only)
ViewModel	UiState transitions, events	coroutines-test, fakes

Layer	Scope	Tech
Compose UI	per-screen flows & states	Compose UI test + Robolectric/emulator
Navigation	route/back/deeplink	Compose Nav test
Firebase Emulator	Security Rules (auth), storage rules, CF, sync	Emulator + rules-unit-tests
E2E	critical journeys	instrumented on emulator (dev data only)

2. Mandatory suites

- **Owner tests:** CRUD work/trip, record attendance, manage drivers/labourers, view reports.
- **Driver tests:** login, view own assigned trips only, (permitted) status confirm via CF.
- **Labourer tests:** login, view own attendance only, cannot modify another's (DENY).
- **Unauthorized access:** role trying forbidden action = Forbidden state; unauthenticated = redirect.
- **Authentication:** sign-in/out, disabled account, expired session, account recovery.
- **Firestore rule tests** (emulator): every (actor × verb × resource) from RBAC matrix → ALLOW/DENY expected.
- **Storage rule tests:** upload/download authorization, content-type/size guards.
- **Notification tests:** token reg, role-scoped delivery, deep link authz.
- **Offline tests:** cached reads, offline write queue, retry/backoff, no fake-success.
- **Sync tests:** success/conflict/rejection reconciliation, no silent overwrite.
- **Accessibility tests:** semantics, contrast (design tokens), TalkBack smoke on critical flows.
- **Error-state tests:** every screen's Error/Empty/Offline/Unauthorized/Forbidden states + retry.

3. Fixture & emulator policy

- Never use production Firebase data as fixtures.
- Never run destructive development tests against production.
- Use Firebase Emulator Suite (Auth+Firestore+Functions+Storage) for local dev & CI.
- Mocks only inside tests; **no production mocks/fakes**.

4. Test data

Domain/fixtures from audited entities; a `testdata` module generating org/drivers/labourers/trips for owner, driver, labourer rule fixtures.

5. CI integration

Runs in pipeline (see `CI-CD-ARCHITECTURE.md`): unit→rules(emulator)→Compose→E2E on emulator with dev project.

6. Verification

PROPOSED. Existing Flutter test suite (27 files) is reference for behavioural coverage; native re-implements coverage per this doc.

08-NATIVE-ANDROID/ANDROID-PERMISSIONS.md

Android Permissions (native)

Status: PROPOSED. Principle: **never request permissions preemptively**; request contextually when the feature needs them; document denial behaviour and settings recovery. (Current Flutter app declares no permissions — VERIFIED; native adds only as features require.)

Register

Permission	Feature	When requested	User explanation	Denial	Permanent denial	Settings recovery	Security note
INTERNET	Firebase Auth/ Firestore/FCM/ Storage	normal capability (not a runtime prompt)	—	—	—	—	required once backend added
POST_NOTIFICATIONS (13+)	FCM push (assignment/announcements)	when user enables notifications (contextual)	"Get alerts for assigned work/attendance"	in-app notification centre still works via Firestore	guide to settings	yes (deep link)	notification payload is never authorization
CAMERA / photo access	capture trip/site/ profile photos (PROPOSED; D-8)	on first capture	explain	manual image input	settings	use photo-picker where no permission	avoid broad storage
Location (fine/coarse)	driver trip GPS tracking	NOT required in v1 — DEFER unless product confirms	—	—	—	—	only add if justified; prefer manual status
Storage/media	backup export/import	none needed (use SAF)	—	—	—	—	avoid broad storage
Foreground service/ exact alarm	scheduled reminders / background sync	only if needed	—	—	—	—	optional

Rules

- Contextual + rationale; minimal; each added permission documented here with denial/permanent-denial/settings-recovery.
- Use SAF and the Android Photo Picker to avoid storage permissions entirely where possible.
- Never request location/CAMERA unless a confirmed product feature needs it (D-1/D-8).

Verification

PROPOSED. Current app = no permissions (VERIFIED). Native list above.

PHASE 0 - CONTRACTS & OFFLINE

08-NATIVE-ANDROID/BACKEND-CONTRACT.md

Backend / API Contract

Status: PROPOSED (Phase 0). Even where Firestore is accessed directly, each operation is specified as a domain contract. Structure per op: operation, actor, authorization, inputs, validation, result, error, side effects, audit event, notification, retry.

1. Contract template

Field	Meaning
Operation	canonical name
Actor	role(s) that may call
Authorization	server enforcement (rules/CF), not client claim
Inputs	typed request
Validation	domain/business rule checks
Result	success payload/state
Error	typed error (auth/forbidden/offline/conflict/validation)
Side effects	secondary writes/CF
Audit event	audit log entry
Notification	FCM (type/audience)
Retry	idempotency + backoff

2. Sample contract table

Operation	Actor	Authorization	Notes
Sign in	any	Auth	session
Get own profile	self	rules self	
Update own profile fields	self	self limited fields	no role/email change
List org users	OWNER/ADMIN	rules org+role	
Manage user/role/status	OWNER	CF only	atomic + audit; no client direct
Create work session	OWNER/ADMIN	rules	server numbering
Open/close session	OWNER/ADMIN	rules	close audited
Create trip in session	OWNER/ADMIN	rules/CF for number	tripNumber CF-authoritative
Assign driver/labour to trip	OWNER/ADMIN	rules	may notify driver
Update trip status	OWNER/ADMIN; driver via CF	CF validates state	notify
Record attendance	OWNER/ADMIN (offline queue)	rules/CF	write + audit
Confirm own attendance (driver/labourer)	self	CF	if D-1
Read own attendance history	self	rules	
View reports	OWNER/ADMIN	rules	driver limited own

Operation	Actor	Authorization	Notes
Backup export / restore	OWNER	CF + Storage	encrypted
Send admin notification	OWNER	CF	
Soft-delete record	OWNER	CF	audited
Account deletion	self/OWNER	CF	audited; guards on owner

3. Cross-cutting behaviours

- Every privileged/CF op writes an `auditLogs` doc (actor, action, target, prev/next, reason).
- Every business op defines whether it emits an FCM notification.
- Retry: idempotency keys on mutating ops; outbox replayed idempotently.
- Errors: return a typed result to ViewModel → UiState (never crash, never raw Firebase to UI).

4. Open decisions feeding contracts

D-1 (driver/labourer ops), D-2 (ADMIN delegation), D-6 (driver status workflow), D-8 (retention/audit depth).
Until resolved, use most-restrictive.

5. Verification

PROPOSED. Mirrors the audited Flutter behavioural "workflow" but expressed as server-enforced domain ops.

08-NATIVE-ANDROID/OFFLINE-SYNC-ARCHITECTURE.md

Offline / Synchronization Architecture

Status: PROPOSED. Explicit decision on source-of-truth per domain. The Flutter app was offline-first/Hive — we do **not** blindly carry that; we decide per domain.

1. Decision framework per domain

Domain	Source of truth	Offline read	Offline write	Sync	Conflict	Last-write	Retry/Queue	Recovery
Users/roles/org	Cloud (Firestore + CF)	cached read only	NO (role/profile mgmt requires server)	stream	server-authoritative	server	no local queue	re-auth/fetch
Drivers/Labourers catalogue	Cloud	cached read only	NO (owner writes online or queued)	stream	server-authoritative (CF)	server	optional queue	refetch
WorkSession/Trips	Cloud (authoritative) + local cache	YES (cache)	YES offline queue for owner	batch + stream	versioned; no silent overwrite	merge w/ last-write-wins only where safe	WorkManager queue	reconcile on reconnect
Attendance	Cloud + audit	YES (cache)	YES owner records offline; worker confirm via CF	queue	preserve both sides; explicit resolution	server validates	queue	reconcile; audit
Notifications	Cloud	cached history	n/a	stream	n/a	server	retry send	refetch

Domain	Source of truth	Offline read	Offline write	Sync	Conflict	Last-write	Retry/Queue	Recovery
Drafts	Local (Room/DataStore)	YES	YES (local)	none	n/a	local	n/a	autosave
Settings	Cloud + local cache	cached	owner online	stream	server	server	no	refetch

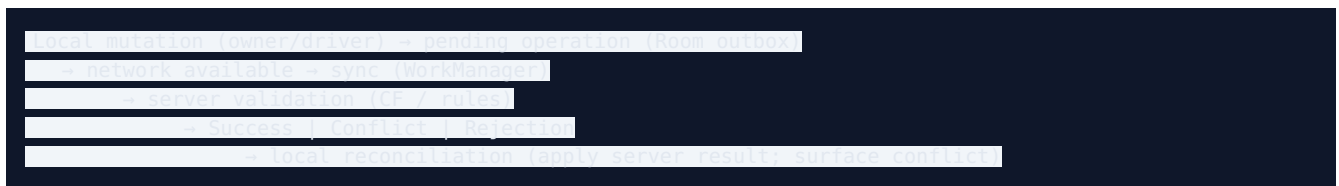
2. Recommended model: Cloud-authoritative with offline-capable owner data entry

- Firestore enabled offline persistence gives cache reads + pending writes.
- Owner is the primary writer; their field entries can be made offline-safe via an explicit offline queue in Room for reliability (rather than relying solely on Firestore pending writes), then replayed by WorkManager.
- Drivers/labourers (if self-service) read own data; any confirmation write is small and can be queued.
- Non-writers (view-only roles) get cached reads + a clear "offline" indicator; no silent success.

3. Offline-first vs Cloud-first conclusion

- **Not offline-authoritative.** Use **hybrid cached state**: Firestore is authoritative for shared/business data; local cache improves offline read and provides a durable offline write queue for owner data entry. Data integrity (trip numbering) must reconcile server-side, not locally.

4. Sync model



- Conflict behavior: use `updatedAt` + revision/version; on divergence, do **not** silently overwrite. Present resolution (keep mine / keep server) for owner-editable records; automated last-write-wins only where safe & documented (e.g., attendance toggle latest by server time + audit).
- Outbox must be transactional with local cache (write local + enqueue in one DB transaction).

5. Retry/backoff & idempotency

- WorkManager with backoff; each queued op has an idempotency key; server rejects duplicate/out-of-order ops (see BACKEND-CONTRACT).

6. Failure & expiry during offline

- Offline sign-in: allowed only if a valid session token exists and rules permit cached auth; otherwise require connectivity. Sessions revoked → Forbidden, reconcile keeps user's unsent outbox clearly labelled (never discard).

7. Data-loss guardrails

- Never drop the outbox on sign-out/account switch without explicit user consent.
- No fake success: if a write is only queued (offline), UI shows "pending/syncing", not "saved to server".

8. Emulator for dev

All sync/conflict logic validated against the Firebase Emulator (never production) before release. See `ANDROID-TESTING-ARCHITECTURE.md`.

9. Verification

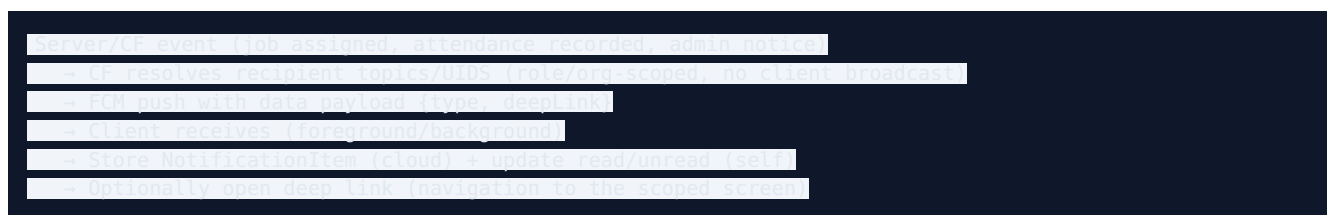
PROPOSED. Ties to ADR-012. Needs product confirmation on conflict UX (D-6/D-1).

08-NATIVE-ANDROID/NOTIFICATION-ARCHITECTURE.md

Notification Architecture (FCM)

Status: PROPOSED. Never treat notification payloads as authorization — data access is still governed by rules.

1. Flow



2. Token registration/refresh/revocation

- Client registers FCM token to its own `users/{uid}/fcmTokens` on login & on `onNewToken`.
- Token refresh handled by FCM SDK; remove stale tokens on push failure (CF).
- Revoked token / sign-out → remove token from the user's doc (no cross-user reuse).
- App Check on all token writes.

3. Topics/subscriptions vs direct UID

- Role/org notifications: subscribe a device to `org_{orgId}` and/or role topic only via server registration rules; never let a client arbitrarily subscribe to another's topic.
- Personal notifications: direct to `users/{uid}` token list (server).
- Never trust a client-supplied recipient list.

4. Notification types (role-scoped)

Type	Audience	Trigger	Deep link
WORK_ASSIGNED	driver (if D-1/6) / labourer	owner assigns	trip/work detail (scoped)
STATUS_UPDATED	owner + relevant	status change	detail
ATTENDANCE_CONFIRMED	labourer (if self)	confirm	attendance
ADMIN_ANNOUNCEMENT	org members	owner	notice
SYSTEM (backup/export ready)	owner	CF job	settings

5. Read/unread & history

- `notifications/{userId}/{id}` server-created; client marks read; history list with unread count; retention window per policy.
- Cleanup of old notifications by CF on schedule.

6. Foreground vs background

- Foreground: show in-app notification center + optional heads-up; no duplicate push if in-app.
- Background/killed: FCM data + notification; tapping deep-links into the scoped screen with auth check.

7. Permission strategy

POST_NOTIFICATIONS (Android 13+) requested contextually when enabling notifications; denial handled gracefully (in-app centre still works via Firestore reads); permanent-denial → settings deep link. Never request preemptively. See `ANDROID-PERMISSIONS.md`.

8. Monitoring

Track FCM delivery (CF analytics on send result), token errors, notification open rate (opt-in). No unnecessary PII.

9. Anti-patterns

- No client-side broadcast; no trusting payload to grant access; no storage of tokens outside secured user docs; no logging of notification content secrets.

10. Verification

PROPOSED; enabled in a later v1.x (ADR-014). Requires POST_NOTIFICATIONS + backend before acceptance.

08-NATIVE-ANDROID/MIGRATION-STRATEGY.md

Migration Strategy (Flutter/Hive → Native/Firebase/Room)

Status: PROPOSED. **DECISION REQUIRED (D-3):** Whether any real production data exists in a deployed Flutter install. The native app is net-new; if no production Flutter data exists, there is **nothing to migrate** and that must be stated explicitly (never silently discard real data).

1. Premise

- The Flutter app is a single-user offline tool with Hive boxes and optional `.labourbackup` exports.
- The native app is a new, cloud-authoritative product. Most current single-user local data would, if imported, need to be assigned an owner/org and re-validated against new invariants (session numbering, catalogue references rather than free-text drivers).

2. Decision matrix

Scenario	Handling
No production Flutter data (likely)	No migration required. State explicitly. Provide a one-time import path if an operator later wants to bring a <code>.labourbackup</code> into their org.
Operator has a <code>.labourbackup</code>	Optional importer: parse → validate → map Hive records to native (org-scoped) entities → human review of ambiguous fields (driver free-text → driver catalogue; trip numbering) → import with audit.
Operator still running Flutter while native launches	Keep Flutter as read-only legacy; no bidirectional sync in v1 (avoid split-brain). Provide export guidance.

3. Import path design (if used)

```
labourbackup_for_native_import
→ Validate (schema, bounds)
→ Transform (map Work-WorkSession, Trip-Trip w/ driver catalogue resolution, Labour-Labourer,
TripLabour-Attendance)
→ review/confirm ambiguous mappings
```

```
-> Import to org Firestore (via OWNER CF, idempotent, audited)
Verification report: success, skipped
```

Never silently discard data: report every skipped record and require confirmation.

4. What NOT to migrate

- Single-user/no-auth assumptions.
- Free-text driver names without a matching catalogue entry (require mapping).
- Draft autosaves (transient).
- Local-only backups as authoritative (import only on explicit user action).

5. Data format notes

Hive `.labourbackup` fields documented in the audit (`STORAGE-ANALYSIS.md` , `DATABASE.md`). Native export format is new (see `BACKUP-RECOVERY /security`): encrypted, signed, versioned.

6. Decision required (D-3)

Confirm existence of production Flutter data & whether import matters. Until then, treat migration as **optional importer (DEFER/off)**; no data-destructive actions.

7. Verification

PROPOSED. No migration code written (Phase 0).

PHASE 0 - PLATFORM & OPS

08-NATIVE-ANDROID/DEPENDENCY-POLICY.md

Dependency Policy

Status: PROPOSED (Phase 0). Rules for choosing and maintaining native dependencies. Purpose-first: every dependency must answer «What problem does this solve?».

1. Rules

- Prefer official AndroidX / Google / JetBrains libraries.
- Prefer **stable** versions; do not adopt alpha/beta merely because version number is larger (e.g., Material 3 stable is **1.4.0**; 1.5.0 is alpha — do not use).
- Verify current stable versions from official release notes **before implementation**; record them in the implementation spec (see Version Strategy below).
- Avoid unnecessary/abandoned libraries; avoid adding for popularity.
- Review transitive dependencies & license/security.
- Document each dependency's purpose.
- Run security scans (dependency audit) in CI.
- Periodically update within stable channels and re-run tests.

2. Proposed dependency set (each with purpose)

Category	Library	Purpose	Justification
DI	Hilt (Dagger)	dependency injection	compile-safe, Jetpack integration
Async	Kotlin coroutines + Flow	concurrency, reactive state	core AndroidX
UI	Jetpack Compose + Material 3 (stable 1.4.0) + Navigation Compose	UI/design/nav	required
Lifecycle	lifecycle-viewmodel-compose, collectAsStateWithLifecycle	state collection	AndroidX
State	kotlinx StateFlow/SharedFlow	UDF state	core
Data	Cloud Firestore (Firebase BoM), Firebase Auth	cloud data/identity	required
Local	Room (only where local relational justified: outbox/cache), DataStore	local	see ADR-011
Images	Coil	async image load/cache	only if photos added
Background	WorkManager	offline queue/backup	reliability
Parsing/ JSON	kotlinx.serialization	DTO mapping	optional (only if needed)
Testing	JUnit, coroutines-test, Compose UI test, MockK/fakes, Robolectric, Firebase Emulator SDKs, rules-unit-testing	tests	required
DI testing	dagger hilt-android-testing	test DI	required
Ops	Crashlytics, (optional) Analytics, App Check	reliability	required

3. Libraries explicitly NOT added (unless justified later)

Retrofit/OkHttp (no bespoke HTTP API — Firestore SDK), Mapbox/Google Maps (no GPS in v1), image-loading unless media added, DI alternatives, multiplatform frameworks.

4. Anti-goal

Do not add libraries to look sophisticated. Add only when it solves a concrete, justified problem.

5. Version Strategy

Record verified stable versions in this section at implementation time (before code). Current environment date: 2026-09. Anchor (verify from official notes at implementation): - Material 3 (androidx.compose.material3): stable **1.4.0** (VERIFIED via androidx release notes). 1.5.0-alpha27 is NOT stable → do not adopt. - Compose BOM: recent stable 2026.x (e.g., 2026.08.00) maps Compose core ~1.12.0 stable; **confirm BOM maps to stable core before use**. - Verify before implementation (mark each): Android Studio, AGP, Gradle, Kotlin, Compose BOM, Compose core, Material3, Navigation Compose, Lifecycle, Hilt, Firebase BoM, Room, WorkManager, Coil, Crashlytics, App Check. - Compose 1.12+ may require compileSdk 37 + AGP 9 (per release notes) → plan toolchain accordingly; do not force an AGP/compileSdk bump unless required by stable deps.

6. Verification

Version values above are from public androidx release notes as of audit date; **re-verify at implementation** before pinning. No dependencies resolved/build yet.

08-NATIVE-ANDROID/OBSERVABILITY-ARCHITECTURE.md

Observability Architecture

Status: PROPOSED (Phase 0). Current Flutter app has no telemetry (VERIFIED). Native adds privacy-conscious observability. Do not collect unnecessary personal data.

1. Layers

Layer	Mechanism	Notes
Crash	Crashlytics	fatal/non-fatal, non-PII breadcrumbs
Structured app logs	Timber-like wrapper with levels	no secrets/PII; debug-only verbose
Auth monitoring	Firebase Auth events (sign-in success/failure, reset)	aggregate; watch for brute force
Backend logs	Cloud Functions logging	redact; structured
Audit logs	<code>auditLogs</code> Firestore (see FIREBASE-DATABASE-DESIGN)	privileged actions only
Notification delivery	FCM send result + CF metrics	token errors, failures
Performance	(optional) Crashlytics custom traces / AndroidX metrics for startup & key flows	privacy-reviewed; do not over-collect

2. Logging rules (mandatory)

Never log: passwords, tokens, auth headers, secrets, full PII (names/phones), private docs, notification content. Use correlation IDs not PII where possible.

3. Analytics

Only if it provides genuine product value AND is opt-in/privacy-compliant (default off). Not added for show.

4. Alerting thresholds

Define for error rate, crash-free sessions, sign-in failures, CF error rate, FCM failure rate. (Values set at implementation/ops with product.)

5. Backend service health

Firestore quota/usage, CF invocation errors, scheduled-job health, FCM. Surfaces to operator/admin reporting (owner).

6. Verification

PROPOSED. Current telemetry absent (VERIFIED).

08-NATIVE-ANDROID/CI-CD-ARCHITECTURE.md

CI/CD Architecture

Status: PROPOSED (Phase 0). No CI exists in the repo (VERIFIED). Design with production secrets in CI secret storage — never commit credentials.

1. Environment strategy

Env	Firebase project	Usage	Rules
development	separate dev Firebase project	dev build, emulator, local CI	debug never points to prod
staging	separate staging project	release-candidate verification, emulator-acceptance	test data only
production	production project	end-user	App Check; no destructive tests

- Bind each build type to the correct Firebase project via `google-services.json` build-type/firebase config. **Debug builds must never connect to production services.**
- Do not invent actual project IDs (D-4); use placeholders in config, verify at provisioning.

2. Pipeline

Build Requests
→ Formatting (ktlint)
→ Static Analysis (detekt/lint/Android Lint)
→ Unit Tests
→ Firebase Emulator (dependency audit)
→ Compose UI Tests
→ Security checks (secret scan gitleaks; dependency audit; SAST)
→ Debug Build
→ Release Build (debug/production)
→ App Check Verification (manual+automated)
→ Staging
→ Acceptance (staging project; emulator; manual+automated)
→ Production (requires explicit approval/owner sign-off)

Production deployment gated on explicit approval (OWNER).

3. Secrets & signing

- Keystore + keystore passwords + Play service account in CI secret store only.
- Play App Signing; upload key not in repo (remediation of audit SEC-1).
- google-services.json for prod not committed in a way that leaks to debug builds.

4. Artifacts & release

- versionCode/versionName automated; release notes; staged rollout; rollback path (prior artifact); post-release crash monitoring.

5. Quality gates

Enforce gate from `06-QUALITY/PRE-RELEASE.md` + emulator rules tests + no-CRITICAL/HIGH open.

6. Verification

PROPOSED. No CI implemented.

08-NATIVE-ANDROID/IMPLEMENTATION-ROADMAP.md

Native Android Implementation Roadmap (Phase 0 → implementation)

Status: PROPOSED. Phase 0 architecture is the gate into implementation. **Do not implement until the Phase-0 gate (§47) passes and open decisions are resolved.** Production release blocked until audit SEC-1 (keystore) remediated.

Phase 0 (this deliverable)

Reconciliation + architecture + Firebase/RBAC/rules design + offline/sync + notifications + security + testing + CI/CD + ADRs. Output: `PRODUCT-RECONCILIATION.md` and the full `08-NATIVE-ANDROID/` set + `ADR/`.

Phase-0 exit criteria (do not start implementation until): - Open decisions D-1..D-8 answered or explicitly flagged. - Version matrix verified (DEPENDENCY-POLICY) on a Flutter-independent machine. - Keystore SEC-1 remediated (rotate + purge). - No DECISION REQUIRED in the docs blocks implementation.

Phase 1 — Foundation & offline core (no backend yet)

Project skeleton (Kotlin/Compose/M3 stable), Hilt, design system, Navigation graphs (auth placeholder gated), Clean Architecture packages, Room outbox+cache, DataStore, local domain entities + repository interfaces + unit/domain tests. Port owner day-entry/trip/attendance behaviour as local-first (KEEP behavioural scope).

Phase 2 — Authentication & roles

Firebase Auth (email/password), provisioning Cloud Function for owner/admin (no hardcoded creds), role model, session/forbidden handling, role-driven nav.

Phase 3 — Backend data & security

Firestore schema + Security Rules + **Emulator rule tests** (mandatory) + App Check; Firestore datasources + mappers; offline/sync reconcile + WorkManager; CF for privileged ops + audit.

Phase 4 — Role journeys & notifications (scope-dependent D-1/D-6)

Driver/labourer (if enabled): role homes, assigned-trip reads, CF-gated status/attendance confirm. Notifications (FCM + CF + token lifecycle) if in v1.

Phase 5 — Reports, backup/DR, observability

Owner reports; encrypted/signed backup/export + cloud backup; Crashlytics + monitoring + alerting; audit tooling.

Phase 6 — Release hardening

Accessibility & localization; performance (macrobenchmark/baseline profiles); release gate (PRE-RELEASE.md); staged rollout; post-release monitoring; Play App Signing.

Cross-cutting

Tests every phase (unit/ViewModel/Compose/Emulator-rules/RBAC/offline/sync/all). Keep docs truthful (AGENT.md). No production mocks/placeholders.

Migration note

Optional importer for `.labourbackup` /Hive only if D-3 confirms production data (MIGRATION-STRATEGY.md).

Verification

Phase 0 spec complete; implementation not started (NOT READY until gate passes).

ADR-001: Kotlin

Context

The native app must be built on a modern, officially supported Android language with strong coroutine/Flow support and Java interop. Alternatives: Java (legacy), Kotlin, (Flutter already used but being replaced).

Decision

Use **Kotlin** for the native Android application.

Why chosen

- First-class Android language; Jetpack/Compose/Firebase/KTX APIs are Kotlin-first.
- Coroutines + Flow align with the UDF/state architecture.
- Null-safety & concise domain modelling (sealed classes for states).
- Official Google/AndroidX + Firebase sample/guidance is Kotlin.

Alternatives

- Java: possible but verbose; lacks Flow/coroutine ergonomics; more boilerplate.
- (Flutter already ruled out for native rebuild.)

Trade-offs / consequences

- Need Kotlin Gradle plugin & toolchain; team must be Kotlin-fluent.
- Interop with any legacy Java code is seamless if needed.
- Version must be verified (DEPENDENCY-POLICY).

ADR-002: Jetpack Compose

Context

Need a modern declarative UI toolkit that scales from phone to tablet, integrates with ViewModel/Flow, and supports a strong design system.

Decision

Use **Jetpack Compose** for all UI.

Why chosen

- Declarative UDF fits ViewModel/StateFlow architecture.
- Material 3 + adaptive layouts support large screens.
- Official stable (Compose core stable; use Compose BOM mapping stable core, verify version).
- Testability (Compose UI tests).

Alternatives

- View/XML: more verbose, imperative, harder to adapt & test.
- Third-party UI kits: not needed, avoid dependency.

Trade-offs / consequences

- Compose learning curve; recomposition discipline required (stable state, keys).
- Some 1.12+ features require compileSdk 37/AGP 9 — keep to stable supported set.
- ADR-003 (Material 3) layered on Compose.

ADR-003: Material 3

Context

Need a coherent, accessible, adaptive design system as the base for a premium product UI.

Decision

Use **Material 3** (stable **1.4.0**) as the foundation, extended with a small custom product design system rather than replacing it.

Why chosen

- M3 is the modern standard; supports theming, dynamic color, adaptive layouts, large screens, accessibility.
- Build a product layer ON TOP (extend/customize color, type, shape, components) instead of reinventing.
- Do not adopt alpha (1.5.0) merely for a higher version number.

Alternatives

- Material 2: older, less adaptive/dynamic.
- Fully custom design system: high cost, no accessibility foundations.
- Third-party design systems: unnecessary dependency.

Trade-offs / consequences

- Follow M3 component + ally conventions; restrict to stable APIs.
- Dynamic color optional behind a setting (respect brand).
- Coordinate with ANDROID-DESIGN-SYSTEM.md.

ADR-004: Clean Architecture

Context

The Flutter reference used a "clean-ish" layered approach but with presentation→data leaks, dead code and business logic in screens. Native needs maintainable, testable boundaries and a backend-independent domain.

Decision

Use **Clean Architecture** layering: Presentation → Domain ← Data, with domain independent of Firebase/Android. Repository interfaces are the seam. Use cases hold business rules.

Why chosen

- Domain testability & independence from Firebase/Room.
- Repository interface enables local/remote/emulator swapping.
- Keeps UI free of backend/business logic.
- Matches audit recommendation; fixes the Flutter smell (presentation→Hive leaks).

Alternatives

- Single-layer "everything in Activity/ViewModel": faster initially, but untestable, unmaintainable, high tech-debt (the very issues flagged).
- Unbounded modularization: over-engineered for this size (avoid).

Trade-offs / consequences

- More files/boilerplate; discipline required (no shortcuts into data from UI).
- Repository + mapper overhead.
- ADR-006 (Hilt) wires the layers.

ADR-005: UDF / MVI-style State Model

Context

Flutter used BLoC with sealed states; native needs explicit, testable, one-way state without arbitrary global mutable state.

Decision

Use **UDF/MVI-style** state management: UI emits intents/events → ViewModel runs use cases → exposes `StateFlow<UiState>` (sealed). States: Initial/Loading/Success/Empty/Error/Offline/Unauthorized/Forbidden/Refreshing/Submitting/SuccessWithData. Navigation/one-shot events handled explicitly (ADR-013), not as data state where avoidable.

Why chosen

- Predictable, single-source-of-truth per screen.
- `collectAsStateWithLifecycle` lifecycle-aware.
- Sealed `UiState` = exhaustive, recoverable handling.
- Testable ViewModels.

Alternatives

- Plain mutable `MutableState` everywhere: error-prone, no explicit transitions.
- Redux: boilerplate-heavy; not needed.
- Single global store: unnecessary.

Trade-offs / consequences

- ViewModels per screen; discipline to keep state in sealed classes; explicit loading/offline/forbidden states required by error-state docs.
- Mapping handled in ViewModel/domain.

ADR-006: Hilt (Dependency Injection)

Context

Flutter used `get_it`. Native needs DI that is compile-safe, integrates with `ViewModel/WorkManager/Instrumentation` and supports test swapping.

Decision

Use **Hilt** (Dagger) for dependency injection.

Why chosen

- Compile-time, AndroidX-integrated (`ViewModel`, `Navigation`, `WorkManager`).
- Test modules can substitute fakes/emulator datasources.
- Aligns with Google/Firebase guidance.

Alternatives

- Manual DI: boilerplate, error-prone.
- Koin: simpler but runtime, weaker compile safety.

Trade-offs / consequences

- Adds `kapt/ksp` processing time; requires Hilt testing setup.
- Keep graph tidy; avoid over-injection.
- ADR-004 layers wired by Hilt.

ADR-007: Firebase as Backend Platform

Context

The Flutter app was fully offline single-user. The product brief requires secure multi-role operation with a backend. Decision is which backend platform.

Decision

Use **Firebase** (Authentication, Firestore, optional Storage, Cloud Functions, FCM, App Check, Crashlytics) as the managed backend platform.

Why chosen

- Managed, serverless; rapid secure app auth + rules + offline persistence.
- Aligns with requested stack and Google guidance (Auth + Security Rules for mobile authorization, App Check).
- Enables offline-capable Firestore + FCM + Cloud Functions without infra.

Alternatives

- Custom REST backend + own DB: high infra/ops cost, slower to market, more security surface.
- Supabase/other BaaS: viable but diverges from the mandated Firebase stack.

Trade-offs / consequences

- Vendor lock-in to Firebase.
- Requires strong Security Rules + App Check + CF discipline (ADR-008..010).
- Remediate audit SEC-1 before any release. Env strategy in CI-CD-ARCHITECTURE.

ADR-008: Cloud Firestore as primary data store

Context

Need a cloud store for org-scoped domain data that supports security rules, offline persistence, realtime-ish updates, and pagination — superseding Hive as the authoritative store.

Decision

Use **Cloud Firestore** as the primary cloud application data store (conceptual schema in FIREBASE-DATABASE-DESIGN.md).

Why chosen

- Rules-based authorization + offline persistence + flexible queries.
- Direct client access under well-tested rules (not CF-proxying every call).
- Live sync for owner/driver/labourer.

Alternatives

- Realtime Database: less flexible queries, more complex permission modeling for this domain.
- Own backend DB: infra cost (ADR-007).
- Hive/Room only: no shared/secure cloud authority.

Trade-offs / consequences

- Denormalization/composite indexes must be planned (documented).
- Query modeling matters; keep ownership keys (orgId) for rules.
- Security Rules are the enforcement boundary (ADR-010).

ADR-009: Authentication architecture

Context

Product brief requires Owner/Admin/Driver/Labourer roles with server-authoritative authorization and **no hardcoded owner credentials** (owner = business identity Ramesh Sahu).

Decision

Use **Firestore Authentication** (email/password for v1; phone evaluated only if D-1 adds driver/labourer field users). Owner/admin identities provisioned server-side (Cloud Function) and bound to role — never hardcoded. Session via Firestore token persistence; App Check attached.

Why chosen

- Meets identity + server-authoritative-role requirement.
- Rules can check `request.auth.uid` and role.
- Password reset/email verification supported.

Alternatives

- Custom auth: insecure, high cost, unnecessary.
- Hardcoded owner account: strictly forbidden.
- Enabling all providers: unnecessary attack surface.

Trade-offs / consequences

- Choose only needed providers (email/password; phone conditional).
- Account status (active/suspended/deleted) enforced in rules/CF.
- Session-expiry/forbidden behaviour defined in FIREBASE-ARCHITECTURE.

ADR-010: RBAC model

Context

Product requires distinct capabilities per role; authorization must be backend-enforced (never client-side `isAdmin`).

Decision

Model roles as OWNER, ADMIN, DRIVER, LABORER. Store authoritative role in `users.role` (and mirror in a custom claim for rules efficiency). Enforce in Firestore/Storage Security Rules + Cloud Functions. Client uses role only for routing/visibility. RBAC matrix in RBAC-DESIGN.md.

Why chosen

- Backend-authoritative role satisfies the security requirement.
- Custom claims + rules give coarse role checks; `users` doc adds richer metadata/audit.
- Role assignment/transfer only via admin Cloud Function (prevents self-escalation).

Alternatives

- Client-side role flag only: insecure, forbidden.
- Rule checks only on `users` doc each time: more reads; combine with claims.

Trade-offs / consequences

- Must keep custom claims + `users.role` in sync (CF does both, audited).
- Ownership keys (`orgId`, `assignee`, `labourerId`) drive all rules.
- Emulator tests required for every `role`x`verb`.

ADR-011: Local persistence strategy

Context

Flutter used Hive as the offline-authoritative store. Native must decide what local relational/persistent storage is justified (Room is not automatic).

Decision

- **Room** only where local relational persistence is genuinely justified: the **offline write outbox/queue** and the **local cache of Firestore data** that benefits from relational/transactional handling. If the only local need is a simple Firestore cache + outbox, Room is used narrowly for those tables.
- **DataStore** for preferences (non-sensitive) and cached feature flags.
- **Encrypted storage** (SQLCipher/Keystore-backed) for any sensitive cached PII where justified (D-8).
- Draft autosave for owner data entry lives locally (Room/DataStore) — KEEP from reference.

Why chosen

- Avoids Room when unneeded (no heavy local relational domain); Firestore is authoritative.
- Outbox needs transactional local writes → Room justified there.
- Encrypt only what is actually sensitive.

Alternatives

- Hive: offline-authoritative single-user, not appropriate for shared cloud data.
- Room for everything: over-engineering; duplicates Firestore.
- DataStore for everything: not suited for relational outbox.

Trade-offs / consequences

- Narrow Room usage; encryption decision pending D-8 privacy.
- Local = cache + outbox, never sole source of truth (except drafts).

ADR-012: Offline / sync strategy

Context

The reference was offline-first/Hive. We must decide source-of-truth per domain and not blindly carry offline-first forward.

Decision

Use a **Cloud-authoritative with offline-capable owner data entry** model: - Firestore offline persistence for cached reads + pending writes. - A durable **Room outbox** for owner data entry (WorkSession/Trips/Attendance) so offline writes are reliable and reconcile via WorkManager — not relying solely on Firestore pending-writes. - View-only roles (driver/labourer reads) use cache with a clear offline indicator; no fake success. - Conflict: never silently overwrite; version/ updatedAt -based; present resolution for owner-editable records; server-validated last-write-wins only where safe & audited. - Draft autosave is local.

Per-domain decision table in OFFLINE-SYNC-ARCHITECTURE.md.

Why chosen

- Field usage (owner on site) needs reliable offline entry → hybrid cached.
- Shared/multi-role data and security need cloud authority.
- Audit + no-silent-overwrite satisfies data-integrity expectations.

Alternatives

- Pure offline-authoritative (Flutter-style): no secure multi-user, no shared truth.
- Pure cloud-only: poor field UX on connectivity loss.

Trade-offs / consequences

- Sync/conflict/reconciliation complexity (must be tested with Emulator).
- Outbox must be transactional with local cache; idempotency keys.
- Non-writers get read-only cache semantics.

ADR-013: Navigation architecture

Context

Flutter used `go_router` with `extra` object passing (fragile, BUG-17) and an orphaned route. Native needs predictable, role-driven navigation with correct Back/logout/session behaviour and adaptive large-screen support.

Decision

Use **Navigation Compose** with: - AuthGraph (splash/onboarding → login → provisioning) and MainGraph (role-based home + bottom destinations) and DetailGraph (pushed editors/details). - Stable **IDs** loaded via repository for detail screens (no whole-object `extra` passing). - Back/logout/session-expiry semantics defined (see NATIVE-NAVIGATION.md). - Adaptive: NavigationRail + two-pane on medium/large via Material window size classes.

Why chosen

- Type-safe, Compose-integrated, testable.
- Fixes Flutter `extra` fragility.
- Role-driven graph + adaptive support.

Alternatives

- `go_router`: Flutter-era, N/A native.
- Manual navigation: error-prone.
- Navigation 3 experimental: not stable — avoid.

Trade-offs / consequences

- Two nav graphs (auth/main) with a role gate.
- Deep links re-validated by rules.
- Keep ≤ 5 bottom destinations (current design 4).

ADR-014: Notification architecture

Context

Flutter has no notifications (VERIFIED). Product may need role-scoped assignment/attendance/admin notifications with read/unread and deep links.

Decision

Use **FCM** driven by **Cloud Functions** (never client broadcast). Server resolves recipients (role/org/personal topics or token lists via CF), stores `notifications/{userId}` history, and client marks read + deep-links with re-authorization. Token registration/refresh/revocation managed to the user's own doc. Enabled in a later v1.x once backend & POST_NOTIFICATIONS are accepted (not v1.0 core if scope trimmed).

Why chosen

- Server-authoritative sends; recipients scoped server-side.
- Payload never grants access (rules still govern).
- Matches role-scoped requirements.

Alternatives

- Local-only notifications: insufficient for cross-role assignment.
- Client direct send: insecure.

Trade-offs / consequences

- Requires CF + token lifecycle + permission strategy (ANDROID-PERMISSIONS.md).
- Foreground/background/killed handling; read/unread retention.
- Not a v1.0 hard dependency if product trims notifications (ADR-014 staged).

ADR-015: Testing strategy

Context

Reference had a broad but un-executed-in-this-env Flutter test suite with gaps. Native must make security-critical Firebase rules, role behaviour, offline/sync, and every screen's states automatically tested.

Decision

Adopt the layered test strategy in ANDROID-TESTING-ARCHITECTURE.md: unit → domain/repository → ViewModel → Compose UI → navigation → **Firestore Emulator (Security Rules mandatory)** → E2E. Rules tests run in CI; never against production. Mocks only inside tests.

Why chosen

- RBAC/security must be proven via emulator rule tests (owner/driver/labourer/unauthorized).
- Offline/sync validated against Emulator.
- Every UiState (incl. offline/forbidden/error) covered.
- Catches orphan requirements/unauthorized operations (Phase-0 §45).

Alternatives

- Manual-only: insufficient for security & regression.
- Unit-only: misses UI/rules/sync.

Trade-offs / consequences

- Emulator infrastructure in CI (Auth+Firestore+Functions+Storage).
- Test matrix large; prioritize critical journeys + security rules.
- No production data as fixtures; no destructive tests against prod.